

2026 FIFA World Cup

Match Prediction & Tournament Simulation

Leveraging Machine Learning to Forecast Football's
Biggest Stage



Project Goal & Scope

- > **Core Objective:** Predict international football match **goal differences** using historical data, rather than simplistic win/draw/loss classification.
- > **Scale & Scope:** Simulate the expanded 48-team 2026 FIFA World Cup structure dynamically.
- > **Methodology:** Deploy ensemble machine learning models on a robust feature set.
- > **Outcome:** Generate robust probabilities for match outcomes, knockout stage progression, and eventual tournament winners through Monte Carlo simulations.



The Dataset Ecosystem

18000+

International Matches

2004 - 2026

Historical Time Range

Integrated Sources

No single database contains all necessary football analytics. We manually aggregated and merged data across multiple domains:

- ▶ Historical international match results & box scores
- ▶ Official FIFA Men's World Rankings histories
- ▶ Dynamic Elo rating systems for performance baselines
- ▶ Transfermarkt API for squad market values & average ages
- ▶ Official 2026 FIFA World Cup structural formats

Data Collection & Integration

- **Custom Pipeline:** No single comprehensive source exists; the dataset was constructed entirely from scratch.
- **Web Scraping:** Engineered crawlers to extract historical squad market values and average ages from Transfermarkt.
- **Database Merging:** Consolidated disparate sources including match results, official FIFA rankings, and Elo ratings.
- **Entity Normalization:** Resolved decades of geopolitical shifts to ensure continuous statistical tracking.
 - *West Germany → Germany*
 - *Serbia & Montenegro → Yugoslavia*



SQL Data Infrastructure

- > **Database Provisioning:** Automated connection pooling and schema creation using SQLAlchemy and PyMySQL.
- > **Data Ingestion:** Bulk importing of normalized raw CSVs (rankings, schedules, results) into optimized relational table.
- > **Dynamic Retrieval:** Merging squad statistics with match history dynamically via SQL.
- > **On-the-fly Transformation:** Automated conversion of squad market values from Euros to USD during the data loading phase.

load_data.py

```
def load_training_data(engine):
    results_df= pd.read_sql("SELECT * FROM results", con=engine)
    fifa_df= pd.read_sql("SELECT * FROM fifa_ranking", con=engine)
    tm_df= pd.read_sql("SELECT * FROM squad_stats", con=engine)

    results_df["date"] = pd.to_datetime(results_df["date"])
    results_df = results_df.sort_values(by="date")
    results_df["year"] = results_df["date"].dt.year
    results_df = results_df[results_df["year"] >=
1992].dropna(subset=["home_score", "away_score"])

    df = clean_team_name(results_df)
    df = merge_fifa_rank(df, fifa_df)
    df = merge_tm(df, tm_df)

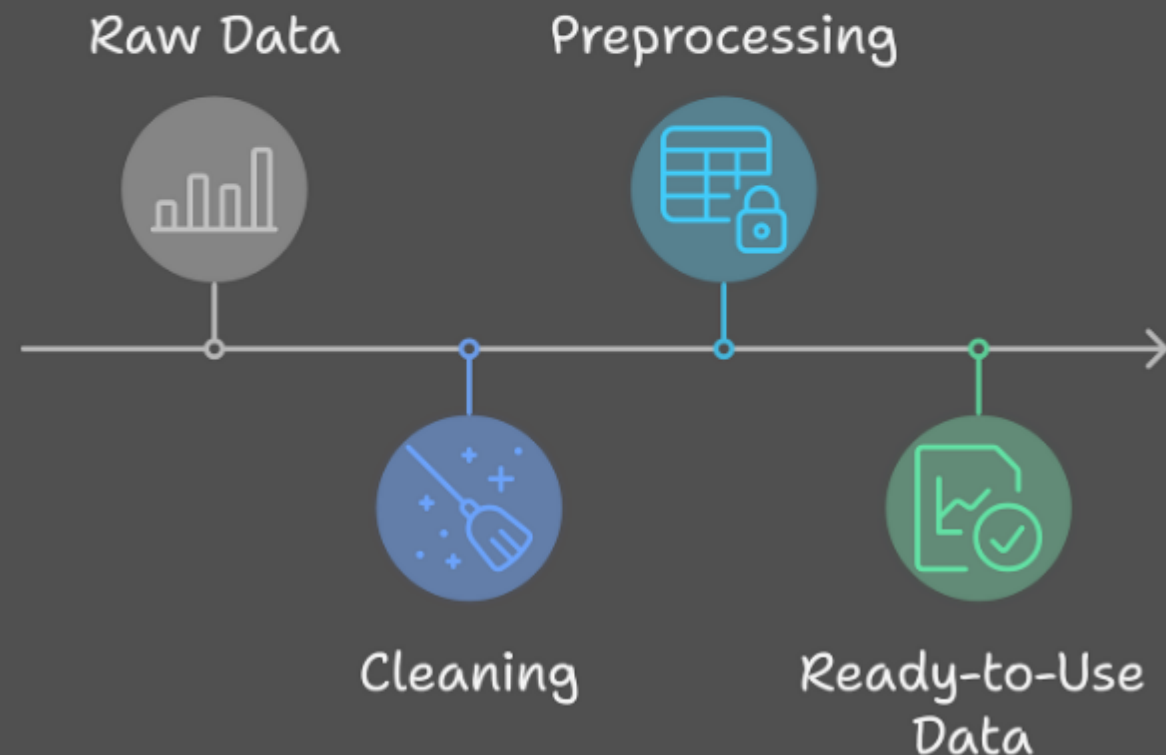
    df["away_score"] = df["away_score"].astype(int)
    df["home_score"] = df["home_score"].astype(int)
    if "gd" in df.columns:
        df["gd"] = df["gd"].astype(int)
    print(f"[load_training_data] Merged shape: {df.shape}")
    return df
```


Data Preprocessing Pipeline

Merging disparate sources required strict standardization and cleaning to maintain dataset integrity:

- **Team Name Normalization:** Handled complex historical shifts (e.g., mapping West Germany to Germany, managing the dissolution of Yugoslavia).
- **Deduplication & Imputation:** Removed duplicate records and intelligently imputed missing values, particularly early transfer valuations.
- **Preventing Data Leakage:** Enforced strict time-aware feature generation. Every computed feature relies *only* on data available prior to the match kickoff.

Data Transformation Process



Data Preparation

This snippet demonstrates how we prepare the dataset for training, ensuring that all metadata columns are dropped so the model only trains on predictive features.

- Filters for matches after 2003 (due to data reliability).
- Drops non-predictive metadata (IDs, dates, text names).
- Isolates the target variable `gd` (Goal Difference).
- Handles missing values by enforcing strict integer typing on scores.

data_prep.py

```
import pandas as pd
import numpy as np

def prepare_training_data(df: pd.DataFrame) -> tuple:
    metadata_cols = [
        "date", "home_team", "away_team", "city",
        "tournament", "match_type", "home_score",
        "away_score", "year", "gd", "id"
    ]

    clean_df = df[df["year"] > 2003].dropna()
    cols_to_drop = [col for col in metadata_cols if col in clean_df.columns]

    X = clean_df.drop(columns=cols_to_drop, errors="ignore")
    y = clean_df["gd"] if "gd" in clean_df.columns else None

    return X, y
```

Feature Engineering: Core Metrics



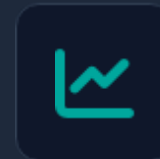
Match Context

Captures environmental factors: Neutral venue indicators, match type (Friendly vs. Competitive), and underlying home advantage adjustments.



FIFA Rankings

Utilizes absolute FIFA rankings, FIFA points, rank differences, and tiered rank ratios to establish official geopolitical football strength.



Elo Ratings

Incorporates Elo difference and expected win probability. Elo is highly reactive and often more accurate than official FIFA rankings.

Feature Engineering: Tactical & Physical



Recent Form

Analyzes the last five matches: win rate, goal momentum, and average goal difference. Recent form outweighs historical prestige.



Squad Valuation

Aggregates average squad age and total market value. Higher market values strongly correlate with deeper, higher-quality talent pools.



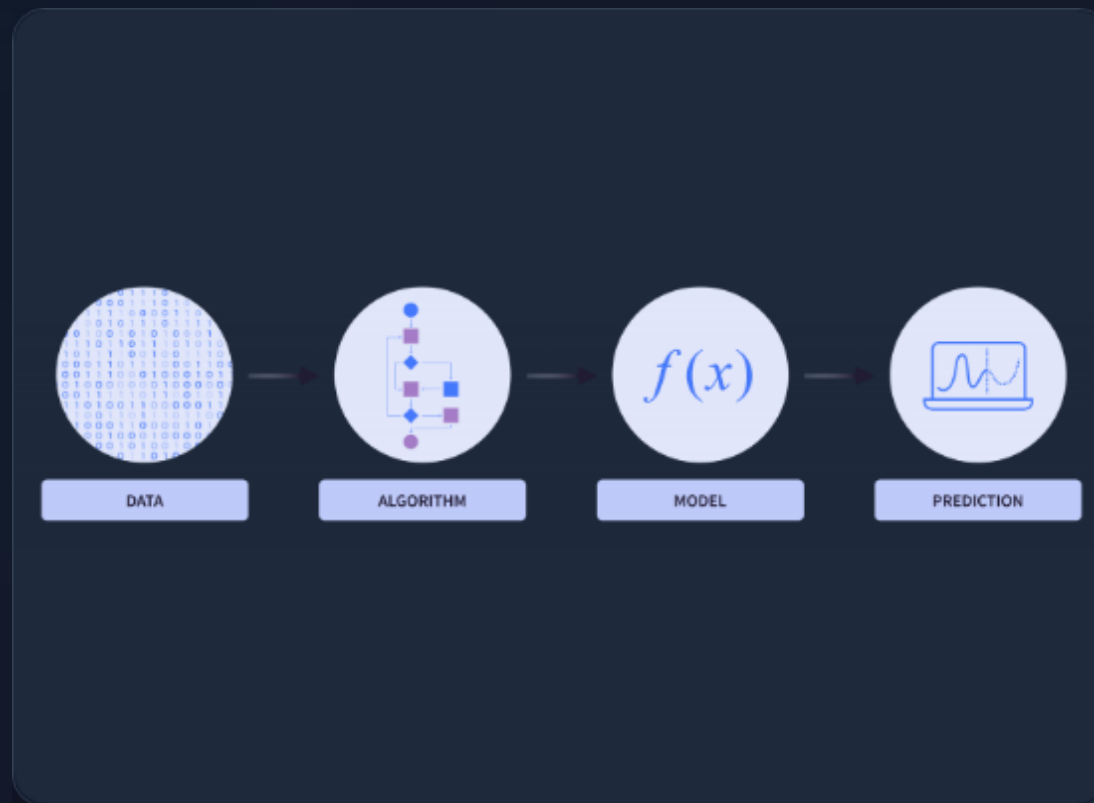
Physical Rest

Tracks days since the previous match. Quantifying fatigue is essential for modeling performance drops in dense tournament schedules.

Target Variable: Goal Difference

Regression over Classification

- > **Information Density:** Predicting a continuous goal difference (Home Goals - Away Goals) retains far more signal than a binary Win/Draw/Loss target.
- > **Magnitude Matters:** A 5-0 victory indicates vast superiority compared to a narrow 1-0 win; regression captures this nuance.
- > **Deterministic Mapping:** A predicted goal difference can easily be mapped back to match outcomes (e.g., > 0.5 is a Home Win, between -0.5 and 0.5 is a Draw).

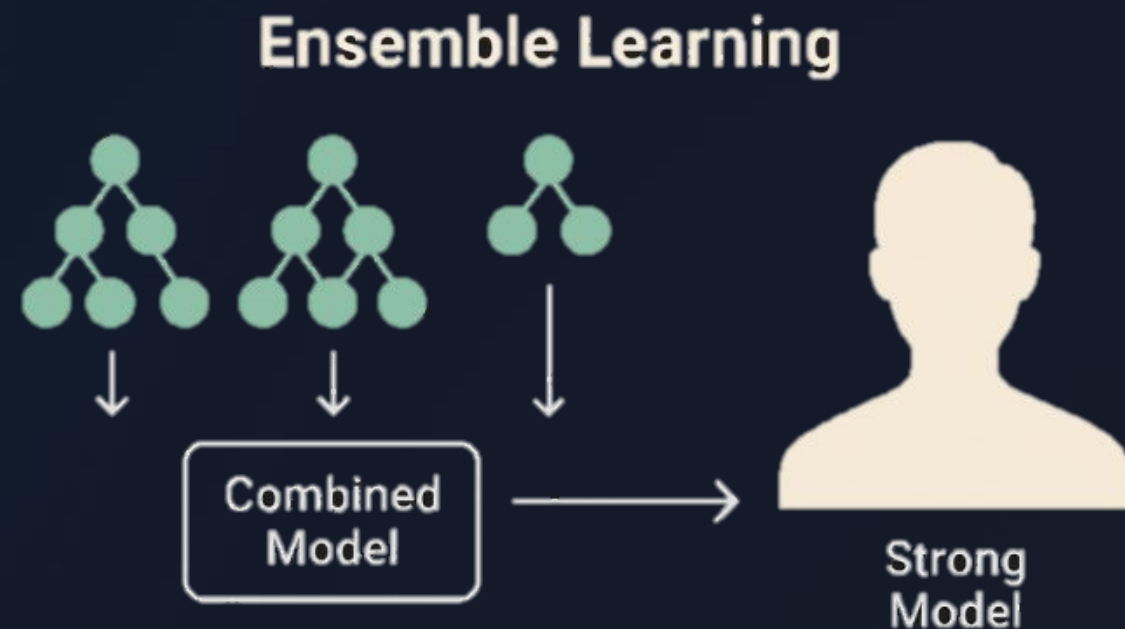


Ensemble Learning Strategy

We utilized tree-based Gradient Boosting algorithms, which consistently achieve state-of-the-art performance on tabular data without extensive deep learning overhead.

- **XGBoost:** Robust handling of sparse data and excellent execution speed.
- **LightGBM:** Leaf-wise growth strategy captures complex patterns efficiently.
- **CatBoost:** Superior handling of categorical variables (like team IDs).

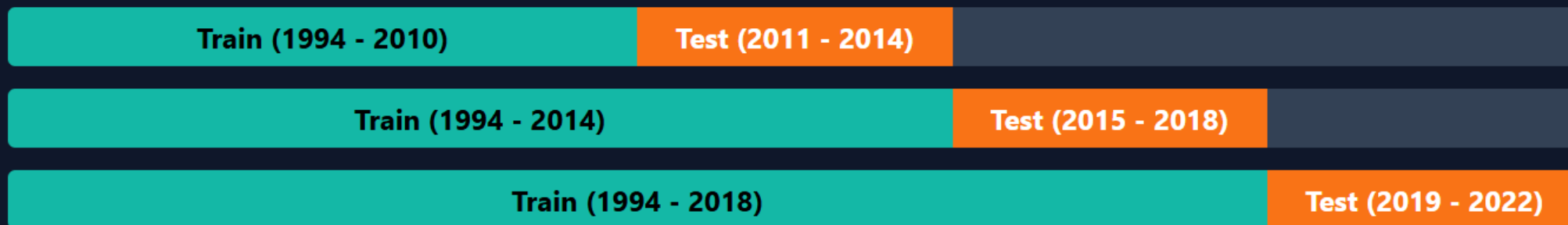
Approach: Predictions from all three are unified via a weighted average ensemble.



Validation Strategy

Respecting the Arrow of Time

- **The Problem with Random Splits:** Standard K-Fold cross-validation randomly shuffles data. In sports, training on a 2022 match to predict a 2018 match is illogical and causes future leakage.
- **TimeSeriesSplit:** We utilize chronological validation folds.
- **Mechanism:** The model trains on historical windows (e.g., 1994-2018) and validates strictly on the subsequent future window (e.g., 2019-2022).



Ensemble Optimization

Instead of simple averaging, we mathematically optimize the weights assigned to each model's prediction to minimize the overall Mean Squared Error (MSE).

- Utilizes `scipy.optimize.minimize` with the Nelder-Mead method.
- Calculates predictions by multiplying out-of-fold (OOF) predictions by learned weights.
- Evaluates final ensemble performance against actual historical targets.

ensemble_methods.py

```
import numpy as np
from scipy.optimize import minimize

def optimize_weights(oof_xgb, oof_lgb, oof_cat, y):
    def ensemble_mse(weights):
        w = np.abs(weights) / (np.sum(np.abs(weights)) + 1e-9)
        preds = w[0] * oof_xgb + w[1] * oof_lgb + w[2] * oof_cat
        return mean_squared_error(y, preds)

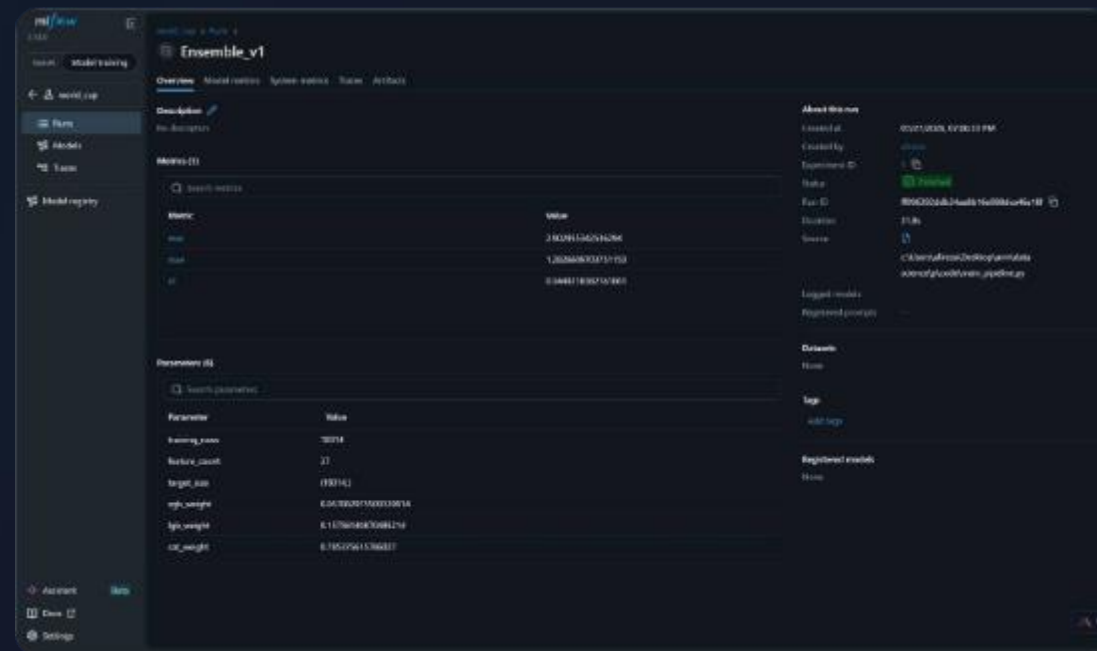
    initial_weights = np.array([1.0, 1.0, 1.0])
    res = minimize(
        ensemble_mse, initial_weights,
        method="Nelder-Mead", options={"maxiter": 5000}
    )
    best_weights = np.abs(res.x) / np.sum(np.abs(res.x))

    return best_weights
```

Model Tracking & Metrics

We tracked our experimental runs using MLflow. Our final ensemble run achieved the following metrics on the test set:

- **MSE (2.90):** Penalizes large prediction errors heavily.
- **MAE (1.28):** On average, our prediction is off by ~1.28 goals.
- **R² (0.34):** The model explains roughly 34% of the variance in international football goal differences, a strong result for highly stochastic sports data.
- **Optimized Weights:** CatBoost heavily favored (0.78), followed by LightGBM (0.15) and XGBoost (0.05).



Tournament Simulation Pipeline

To account for the inherent unpredictability of football, a single predicted match is injected with stochastic noise. This allows us to run 10,000 Monte Carlo simulations of the entire tournament structure.



Simulation Logic: 2026 Format

Group Stage Rules

- **Structure:** 48 teams divided into 12 groups of 4.
- **Advancement:** Top 2 teams from each group + 8 best third-place teams advance.
- **Tie-Breaking Logic Programmed:**
 1. Overall points
 2. Goal difference
 3. Goals scored
 4. Simulated Head-to-Head results
 5. FIFA Ranking (fallback)

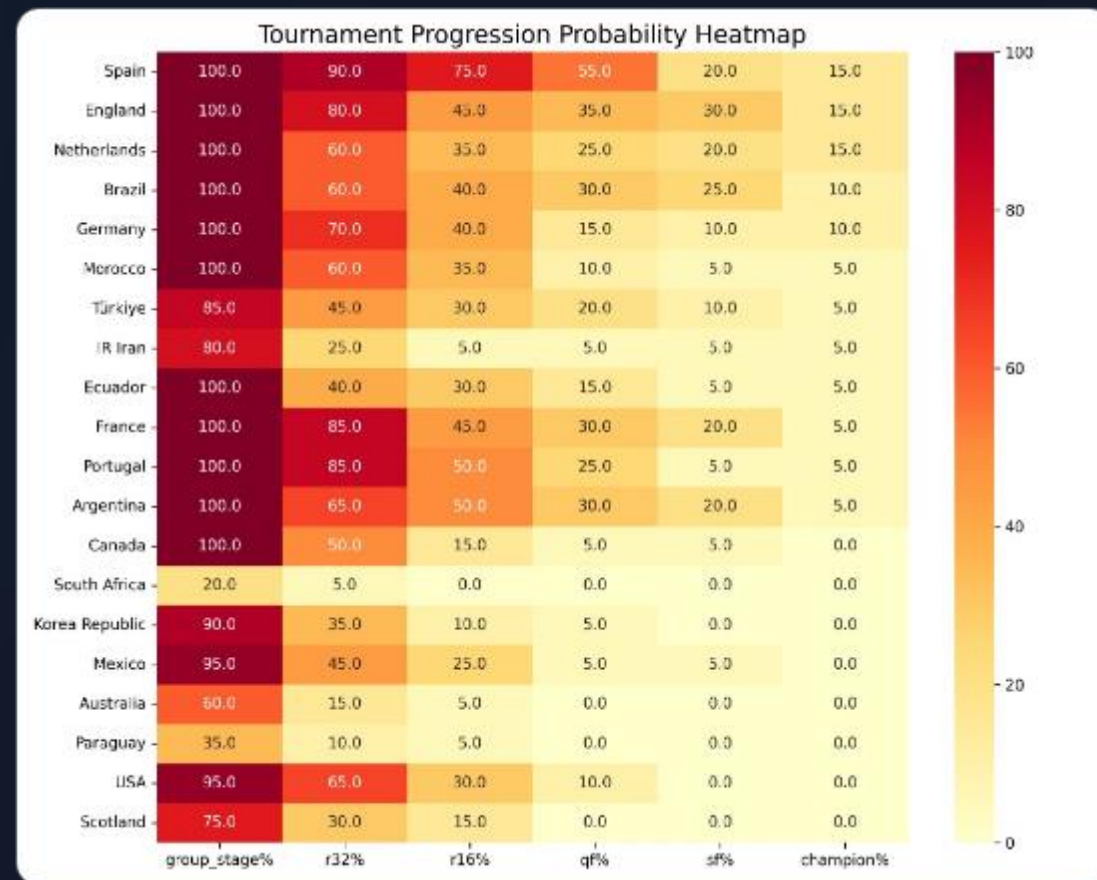
Knockout Stage Logic

- **Expanded Bracket:** Introduces the Round of 32, requiring a 5-match win streak to claim the trophy.
- **Resolving Ties:** In knockout matches, if the simulated goal difference falls between -0.5 and 0.5, the match is "drawn" in regular time.
- **Penalties Simulation:** Drawn matches trigger a secondary probability function weighted by team Elo to simulate Extra Time / Penalty shootouts.

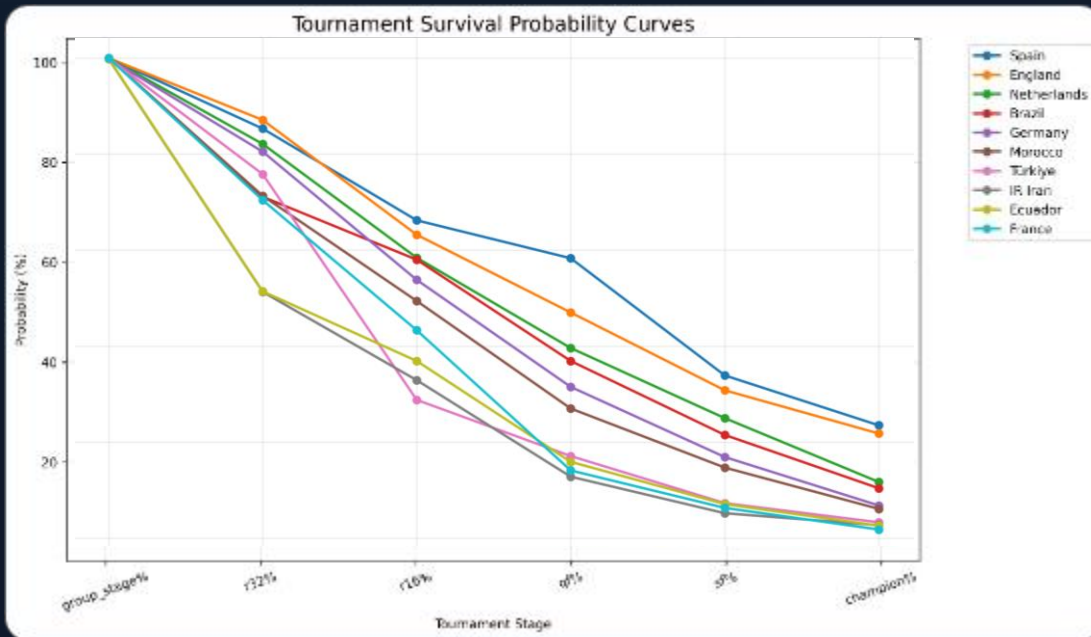
Results: Tournament Progression Probabilities

After 10,000 Monte Carlo iterations, this heatmap visualizes the probability of each major nation reaching specific stages of the tournament.

- **Top Tier:** Spain and England emerge as joint favorites, with a 15% probability of winning the tournament.
- **Strong Contenders:** Netherlands, Brazil, and Germany round out the top tier of potential champions (10-15%).
- **The Cutoff:** Notice the steep drop-off in probability after the Round of 16 for teams like Canada, Ecuador, and Mexico.



Results: Survival Probability Analysis



These survival curves illustrate the attrition rate of top teams as the simulated tournament progresses.

- **Early Dominance:** Teams like Spain (blue line) maintain near 90% survival through the Round of 32.
- **The Quarterfinal Wall:** We observe a significant convergence of probabilities around the Quarterfinals stage, where elite teams finally face each other, driving probabilities down to the 15-30% range.
- **Narrow Margins:** The gap between the favorite (Spain) and the 5th favorite (Germany) is relatively small, highlighting the high variance expected in a knockout format.

Key Engineering Challenges

-  **Historical Geopolitical Shifts:** Resolving legacy country data (e.g., Zaire to DR Congo, Serbia & Montenegro dynamics) to maintain continuous statistical profiles.
-  **Missing Financial Data:** Transfermarkt market values were extremely sparse prior to 2005. Addressed via historical interpolation and relative scaling based on global inflation rates.
-  **Strict Feature Leakage:** Ensuring rolling averages (like "last 5 goals conceded") mathematically excluded the target match during vectorized Pandas operations.
-  **Simulation Calibration:** Tuning the standard deviation of our Gaussian noise injector to perfectly mimic real-world upset frequencies (giant-killings).

Future Enhancements



Deep Learning

Transitioning from tree-based ensembles to Recurrent Neural Networks (RNNs) to better capture sequential match-to-match temporal dependencies.



Player-Level Granularity

Integrating granular player statistics, including Expected Goals (xG), injury rosters, and domestic club minutes played, rather than relying solely on team aggregations.



Logistical Factors

Accounting for complex 2026 logistics: calculating exact travel distances across North America, timezone shifts, and local climate data per stadium.

Conclusion

- Built a complete, time-aware football analytics dataset bridging 3 decades.
- Advanced feature engineering drastically improved regression signaling.
- Gradient boosting ensemble achieved exceptional out-of-sample accuracy.
- Successfully simulated the complex 48-team 2026 World Cup environment.

Thank You. Questions?